

Point-to-Prompt

"질문 모드" — 클릭하면 용어가 프롬프트가 된다

용어를 몰라 프롬프트를 못 짜는 바이브 코더를 위해, hot-reload 렌더 뷰에서 화면 요소를 클릭하면 그 부분의 이름이 **자연어로 프롬프트에 자동 삽입**된다. 지식이 없어도 "무엇을" 가리킬 수 있게 만드는 어휘 다리(vocabulary bridge).

질문 모드 ON → URL 창 클릭 →

프롬프트:  URL 입력 필드 ...를 더 크게 만들고 자동완성 기능을 넣어줘

요소 선택 오버레이

접근성 트리 기반 라벨

이중 표현(사람↔AI)

소스 위치 매핑

Hot-reload 연동

※ 기존 보안 아키텍처 기획은 전면 폐지됨. 본 문서가 대체 기획임 · 2026-07-20

01 문제 재정의 — 진짜 장벽은 "조작"이 아니라 "이름"

바이브 코더는 무엇을 원하는지는 말로 표현할 수 있다("더 크게, 위로"). 못 하는 것은 그 대상을 뭐라 부르는지다.

🤔 현재의 벽

"저 위에 있는... 그거... 주소 넣는 칸?" — 대상의 명칭(URL 입력 필드, 네비게이션 바, 모달 등)을 몰라 프롬프트 첫 단어부터 막힌다. AI는 "어디?"를 되묻고, 대화가 걸돈다.

💡 해법: 손가락으로 가리키기

화면에서 그 부분을 클릭하면 시스템이 대신 이름을 붙여 프롬프트에 넣어준다. 사용자는 이어서 **의도만** 자연어로 쓰면 된다. 용어 지식이 0이어도 진행 가능.

02 핵심 컨셉 & 기존 도구와의 차별점

Onlook·v0·Lovable 등은 이미 "화면에서 클릭→편집"을 한다. 그러나 그들은 **디자인 편집기**다. 우리는 **프롬프트 보조기**다 — 이 차이가 전부다.

	기존 비주얼 에디터 (Onlook / v0 / Lovable)	Point-to-Prompt (본 기획)
클릭의 목적	요소를 직접 조작(드래그·스타일 패널)	요소를 명명해 프롬프트에 넣기 → 자연어로 이어감
전제 지식	디자인 컨트롤 조작법·무엇을 바꿀지 알아야 함	0 — 가리키고 말로 설명만 하면 됨
산출물	즉시 코드 패치	편집 가능한 선택 토큰(칩) + 사용자가 이어 쓰는 프롬프트
타겟 유저	디자이너·개발자	용어를 모르는 비개발 바이브 코더

차별화 한 줄: 기존 도구는 "어떻게 바꿀지 아는 사람"을 위한 편집기, 우리는 "무엇인지 이름조차 모르는 사람"을 위한 **어휘 다리**다. 편집을 대체하지 않고 **프롬프트 작성**을 가능하게 한다.

03 동작 방식 (UX 플로우)



04 핵심 아이디어 — 하나의 클릭, 두 개의 표현

이 제품의 IP는 "선택 토큰(Selection Token)"이다. 사람이 읽는 자연어 라벨과 AI가 읽는 정밀 앵커를 동시에 담는다.

Selection Token — 클릭 한 번의 산출물

humanLabel

USER 프롭프트에 보임

"URL 입력 필드"

role / name

allly 트리

textbox / "주소 입력"

visualPos

"상단 중앙"

selector

AI 숨김 컨텍스트

header nav input.url-bar

sourceLoc

AI 숨김 컨텍스트

AddressBar.tsx:42:6

componentName

<AddressBar/>

screenshot

해당 영역 크롭(비전 모델용, 선택)

왜 "이중 표현"이 결정적인가

사용자는 "URL 입력 필드"만 본다(친숙·안심). 하지만 AI는 뒤에서 파일·라인·셀렉터까지 받는다. 그래서 "어디 말하는 거예요?" 되묻기가 사라지고, **정확한 요소를 한 번에 수정한다.**

효과: 초보자에겐 "말로 가리키기"의 쉬움을, AI에겐 "좌표가 찍힌" 정밀함을 동시에 준다. 이 분리가 제품의 해자.

05 기술 아키텍처

렌더 뷰 계측 → 요소 포착 → 라벨 생성(이중 표현) → 프롬프트 통합 → 에이전트 전달 → hot-reload.

A. 렌더 뷰 계측 (Instrumentation)

hot-reload 개발 서버의 프리뷰(iframe/webview)에 오버레이 스크립트 주입. 마우스 이벤트를 가로채 호버 하이라이트·클릭 포착. 동일 출처(dev server)라 iframe 접근 가능.

B. 라벨 생성 파이프라인 — 저비용 우선, 필요 시 LLM

단계	방법	예시 결과
① 결정론적(무료)	접근성 트리 role + accessible name + 시맨틱 태그 + 인접 라벨 텍스트 → 템플릿 "{위치}의 {name} {role}"	textbox + placeholder "https://" → "URL 입력 필드"
② 소스 힌트	babel 플러그인(__source)· data-oid 로 컴포넌트명 획득	<AddressBar/> → "주소창"
③ LLM/비전 폴백	①②로 모호할 때(의미없는 div) 크롭 스크린샷+DOM 스니펫을 비전 모델에 → 자연어 구	배너 이미지 → "상단 히어로 배너"

대부분은 ①에서 무료로 해결 → LLM 호출 최소화(비용·지연 절감).

C. 소스 위치 매핑 (AI용 앵커)

- 개발 시(소스 有): babel 플러그인이 JSX에 file:line:col 주입 — react-dev-inspector·click-to-component 방식. 가장 정밀.
- 런타임만(소스 無): CSS 셀렉터 + 텍스트 + 스크린샷으로 폴백. 에이전트가 매칭해 수정.

D. 프롬프트 통합 & 에이전트 전달

- 입력창에 칩(토큰)으로 삽입 — 화면엔 라벨만, 내부엔 토큰 JSON 보관.
- 전송 시 사용자 문장 + 숨은 앵커(파일·라인·셀렉터)를 함께 에이전트에 전달.
- 에이전트는 앵커로 정확한 코드 지점을 편집 → hot-reload 반영.

06 프레임워크별 소스 매핑 전략

대상	매핑 방법	정밀도
React (Vite/Next)	@babel/plugin-transform-react-jsx-source 또는 data-oid 주입(Onlook식)	파일:라인:컬럼 (최상)
Vue / Svelte	컴파일러 혹은 소스 위치 속성 주입(플러그인 필요)	파일:라인
순수 HTML/JS	소스 위치 없음 → CSS 셀렉터 + 텍스트 앵커	셀렉터 기반(중)
프로덕션/미니파이	소스맵 있으면 역매핑, 없으면 셀렉터+스크린샷 폴백	가변

MVP는 정밀도가 가장 높고 바이브 코딩 수요가 큰 React(Vite) 단일 타겟으로 시작.

07 개발 로드맵

- Phase 0 — 클릭→칩 최소 루프 · 2~3주

 - 프리뷰 오버레이(호버 하이라이트 + 클릭 포착)
 - 결정론적 라벨(접근성 트리)만으로 칩 생성 → 프롭트 입력창 삽입
 - 앵커는 CSS 셀렉터+텍스트(소스매핑 전) → 에이전트가 매칭 수정
 - React(Vite) 단일 타겟. 여기까지가 데모의 핵심
- Phase 1 — 정밀 소스 매핑 + 정밀도 조절 · 3~4주

 - babel 플러그인으로 `file:line` 앵커 주입 → 에이전트 정확도 급상승
 - 부모 확장(브레드크럼)·다중 선택·칩 편집
- Phase 2 — LLM/비전 폴백 + 라벨 품질 · 3~4주

 - 모호한 요소(무의미 div)에 크롭 스크린샷→비전 모델로 자연어 라벨
 - 한국어 라벨 자연스러움 튜닝, 사용자 수정 학습
- Phase 3 — 프레임워크 확장 · 이후

 - Vue/Svelte/순수 HTML, 이후 앱(모바일 웹뷰) 대상 확대

08 핵심 난제 & 대응

① 선택 정밀도(granularity)

클릭이 leaf에 맞음 → 스크롤 확장·브레드크럼·하이라이트 확인으로 원하는 레벨 선택.

② 라벨 자연스러움

role만으론 "textbox"처럼 딱딱 → placeholder·라벨·위치·컴포넌트명 결합 + LLM 다듬기. 사용자 수정 허용.

③ 앵커 안정성(코드 변경)

hot-reload로 라인 변동 → 소스 재계측 + 셀렉터/텍스트 다중 앵커로 견고화.

④ iframe/Shadow DOM·크로스오리진

동일 출처 dev server 주입 전제. shadow DOM은 composedPath로 관통.

09 착수 전 결정 3가지

① 호스팅 형태

기존 에이전트(Claude Code)에 프리뷰 패널로 엮기 vs 독립 앱

▶ 권장: 엮기

② MVP 타킷

다중 프레임워크 vs React(Vite) 우선

▶ 권장: React 우선

③ 라벨 엔진

처음부터 LLM vs 접근성 트리 결정론 우선

▶ 권장: 결정론 먼저, LLM 폴백

권장 MVP 한 줄: React(Vite) 프리뷰에 오버레이를 엮고 · 접근성 트리로 무료 라벨을 만들어 칩으로 삽입 · 앵커는 셀렉터로 시작 → "URL 창 클릭 → '📌 URL 입력 필드' 자동 삽입 → 이어서 프롬프트" 데모부터 완성.

10 참고 (검증된 기술 토대)

Onlook — data-oid로 DOM↔소스 매핑하는 오픈소스 React 비주얼 에디터
(github.com/onlook-dev/onlook)

react-dev-inspector — 클릭으로 컴포넌트→IDE 소스 점프 (zthxxx)

click-to-component — Option+Click으로 소스 열기, @babel/plugin-transform-react-jsx-source 기반 (ericclemmons)

Accessible Name & Description Computation 1.1 — role+name 산출 (W3C)

Chrome DevTools Accessibility Tree — role/name/state 추출 참고

Point-to-Prompt (질문 모드) · Product Spec v2 · 2026-07-20 — 이전 보안 아키텍처 기획은 폐지되어 본 문서로 대체됨.